



Python Sets — Beginner Notes

1 What is a Set?

A **set** is:

- Unordered (no indexing)
- No duplicate values
- Mutable (can add/remove items)

```
st = {10, 20, 45, 89}
```

✗ No indexing

```
print(st[0]) # TypeError: 'set' object is not subscriptable
```

Sets do **not support indexing** because they are unordered.

2 Creating Empty Collections

```
st = set() # empty set
```

```
lst = list() # empty list
```

```
d = dict() # empty dictionary
```

```
t = tuple() # empty tuple
```

⚠ Important:

```
st = {} # This creates a dictionary, NOT a set
```

3 Updating a Set

```
st = {10, 20}

st.update((100, 1))

print(st)
```

update():

Adds multiple elements from iterable (list/tuple/set).

4 Iterating Over a Set

```
for num in st:

    print(num)
```

Order may change every run because sets are unordered.

5 List vs Set (Conceptual)

Type	Concept
List	Like an array (ordered, allows duplicates)
Set	Like a hashset (unordered, unique values)

6 Removing Duplicates Automatically

```
colors = {"red", "green", "blue", "blue", "yellow"}
```

```
for c in colors:
```

```
    print(c)
```

Output will contain only **one "blue"**.

7 Common Set Methods

Built-in methods (simplified)

Method	Purpose
<code>add()</code>	Add one item
<code>union()</code>	Combine sets
<code>intersection()</code>	Common elements
<code>difference()</code>	Remove overlaps

`issubset()` Check subset

`issuperset()` Check superset

You can see all methods using:

```
print(dir(set))
```

8 Adding Elements

```
foods = {"dosa", "tea", "coffee", "friedrice"}
```

```
foods.add("cb")
```

```
print(foods)
```

`add()` adds a single value.

9 Set Operations

Example Sets

```
set_a = {10, 20, 30, 40, 50}
```

```
set_b = {40, 50, 400, 500, 600}
```

◆ Union (all elements)

```
union_set = set_a.union(set_b)
```

```
print(union_set)
```

♦ Intersection (common elements)

```
i_set = set_a.intersection(set_b)  
print(i_set)
```

Output: {40, 50}

♦ Difference (only in first set)

```
difference_set = set_a.difference(set_b)  
print(difference_set)
```

Output: {10, 20, 30}

10 Subset & Superset

```
set3 = {10, 20, 30, 40, 50}
```

```
set4 = {30, 40}
```

Superset

```
print(set3.issuperset(set4)) # True
```

Set3 contains all elements of Set4.

Subset

```
print(set4.issubset(set3)) # True
```

Set4 is inside Set3.



Key Takeaways

- Sets are unordered and unique
 - No indexing allowed
 - Useful for:
 - Removing duplicates
 - Fast membership checks
 - Mathematical set operations
-